2. Sound and versatile language
3. Fast program execution.
4. An extendible language.
5. Tends to be a structured language.

Historical developments of C(Background)

Year	Language	Developed by	Remarks
1960	ALGOL	International committee	Too general, too abstract
1967	BCPL	Martin Richards at Cambridge university	Could deal with only specific problems
1970	B	Ken Thompson at AT & T	Could deal with only specific problems
1972	C	Dennis Ritche at AT & T	Lost generality of BCPL and B restored

General Structure of a C program:

```
/* Documentation section */
/* Link section */
/* Definition section */
/* Global declaration section */
main()
{
Declaration part
Executable part (statements)
}
/* Sub-program section */
```

➢ The documentation section is used for displaying any information about the program like the purpose of the program, name of the author, date and time written etc, and this section should be enclosed within comment lines. The statements in the documentation section are ignored by the compiler.

➢ The link section consists of the inclusion of header files.

- The definition section consists of macro definitions, defining constants etc,.
- Anything declared in the global declaration section is accessible throughout the program, i.e. accessible to all the functions in the program.
- main() function is mandatory for any program and it includes two parts, the declaration part and the executable part.
- The last section, i.e. sub-program section is optional and used when we require including user defined functions in the program.

First C Program

Before starting the abcd of C language, you need to learn how to write, compile and run the first c program.

To write the first c program, open the C console and write the following code:

```
1. #include <stdio.h>
2. #include <conio.h>
3. void main(){
4. printf("Hello C Language");
5. getch();
6. }
```

#include <stdio.h> includes the **standard input output** library functions. The printf() function is defined in stdio.h .

#include <conio.h> includes the **console input output** library functions. The getch() function is defined in conio.h file.

void main() The **main() function is the entry point of every program** in c language. The void keyword specifies that it returns no value.

printf() The printf() function is **used to print data** on the console.

getch() The getch() function **asks for a single character**. Until you press any key, it blocks the screen.

C TOKENS: The smallest individual units are known as tokens. C has six types of tokens.

1: Identifiers

2: Keywords

3: Constants

4: Strings

5: Special Symbols

6: Operators

Identifiers:

Identifiers refer to the names of variables, constants, functions and arrays. These are user-defined names is called Identifiers. These identifier are defined against a set of rules.

Rules for an Identifier

1. An Identifier can only have alphanumeric characters(a-z , A-Z , 0-9) and underscore(_).
2. The first character of an identifier can only contain alphabet(a-z , A-Z) or underscore (_).
3. Identifiers are also case sensitive in C. For example *name* and *Name* are two different identifier in C.
4. Keywords are not allowed to be used as Identifiers.
5. No special characters, such as semicolon, period, whitespaces, slash or comma are permitted to be used in or as Identifier.
6. C‟ compiler recognizes only the first 31 characters of an identifiers.

Ex :	**Valid**	**Invalid**
	STDNAME	Return
	SUB	*$stay*
	TOT_MARKS	1RECORD
	_TEMP	STD NAME.
	Y2K	

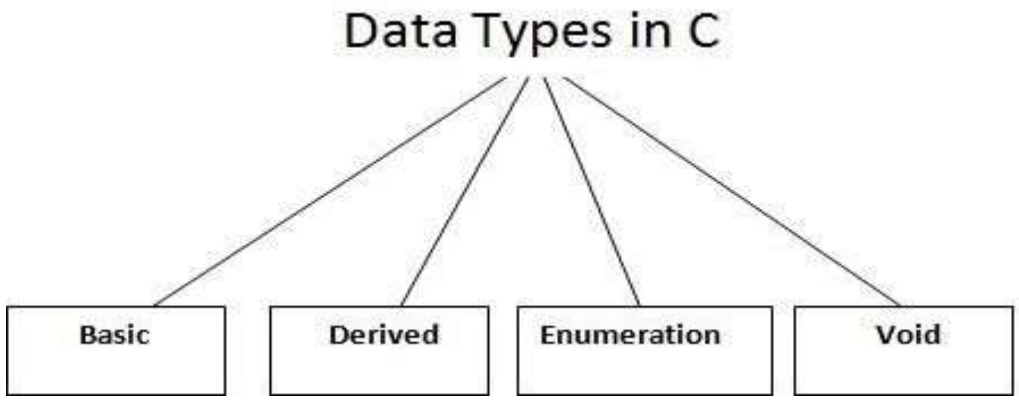
Keywords: A keyword is a **reserved word.** All keywords have fixed meaning that means we cannot change. Keywords serve as basic building blocks for program statements. All keywords must be written in lowercase. A list of 32 keywords in c language is given below:

auto	**break**	**case**	**char**
const	**continue**	**default**	**do**
double	**enum**	**else**	**extern**
float	**for**	**goto**	**if**
int	**long**	**return**	**register**
signed	**short**	**static**	**sizeof**
struct	**switch**	**typedef**	**union**
unsigned	**void**	**volatile**	**while**

Note: Keywords we cannot use it as a variable name, constant name etc.

Data Types/Types:

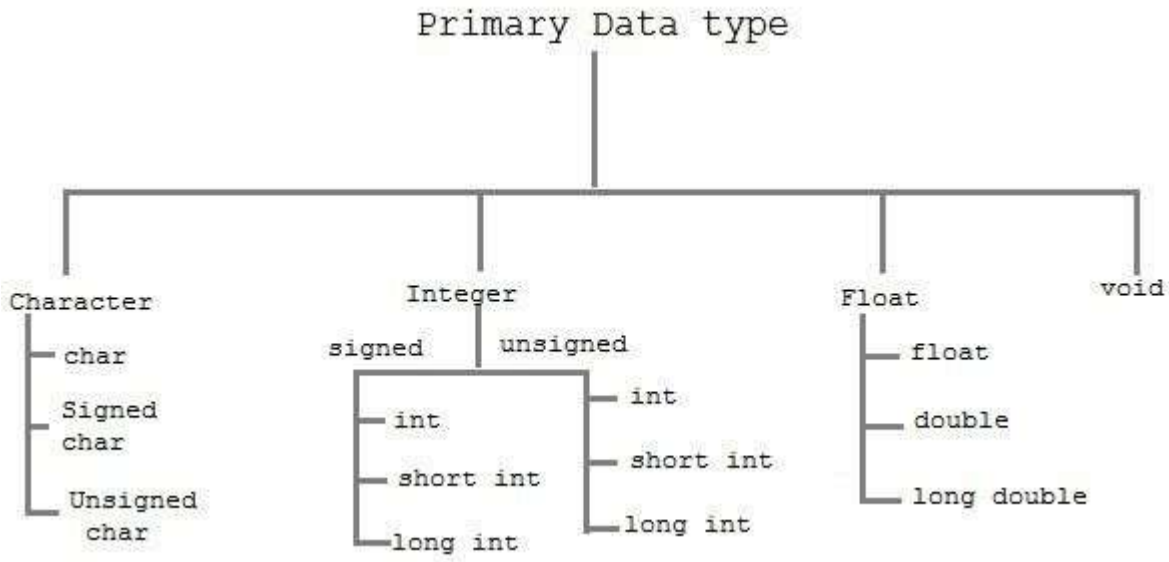
- To store data the program must reserve space which is done using datatype. A datatype is a keyword/predefined instruction used for allocating memory for data. A data type specifies the type of data that a variable can store such as integer, floating, character etc. It used for declaring/defining variables or functions of different types before to use in a program.



There are 4 types of data types in C language.

Types	Data Types
Basic Data Type	**int, char, float, double**
Derived Data Type	**array, pointer, structure, union**
Enumeration Data Type	**enum**
Void Data Type	**void**

Note: We call Basic or Primary data type.



The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals. The memory size of basic data types may change according to 32 or 64 bit operating system. Let‟s see the basic data types. Its size is given **according to 32 bit architecture**.

Size and Ranges of Data Types with Type Qualifiers

Type	Size (bytes)	Range	Control String
char or signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c

int or signed int	2	-32768 to 32767	%d or %i
unsigned int	2	0 to 65535	%u
short int or signed short int	1	-128 to 127	%d or %i
unsigned short int	1	0 to 255	%d or %i
long int or signed long int	4	-2147483648 to 2147483647	%ld
unsigned long int	4	0 to 4294967295	%lu
float	4	3.4E-38 to 3.4E+38	%f or %g
double	8	1.7E-308 to 1.7E+308	%lf
long double	10	3.4E-4932 to 1.1E+4932	%Lf

Variables

A **variable** is a name of memory location. It is used to store data. Variables are changeable, we can change value of a variable during execution of a program. . It can be reused many times.

Note: Variable are nothing but identifiers.

Rules to write variable names:

1. A variable name contains maximum of 30 characters/ Variable name must be upto 8 characters.
2. A variable name includes alphabets and numbers, but it must start with an alphabet.
3. It cannot accept any special characters, blank spaces except under score(_).
4. It should not be a reserved word.

Ex : i rank1 MAX min Student_name StudentName class_mark

Declaration of Variables : A variable can be used to store a value of any data type. The declaration of variables must be done before they are used in the program. The general format for declaring a variable.

Syntax : data_type variable-1,variable-2, ----- , variable-n;
Variables are separated by commas and declaration statement ends with a semicolon.

Ex : int x,y,z;
float a,b;
char m,n;

Assigning values to variables : values can be assigned to variables using the assignment operator (=). The general format statement is :

Syntax : variable = constant;

Ex : x=100;
a= 12.25;
m="f";

we can also assign a value to a variable at the time of the variable is declared. The general format of declaring and assigning value to a variable is :

Syntax : data_type variable = constant;

Ex ; int x=100;
float a=12.25;
char m="f";

Types of Variables in C

There are many types of variables in c:

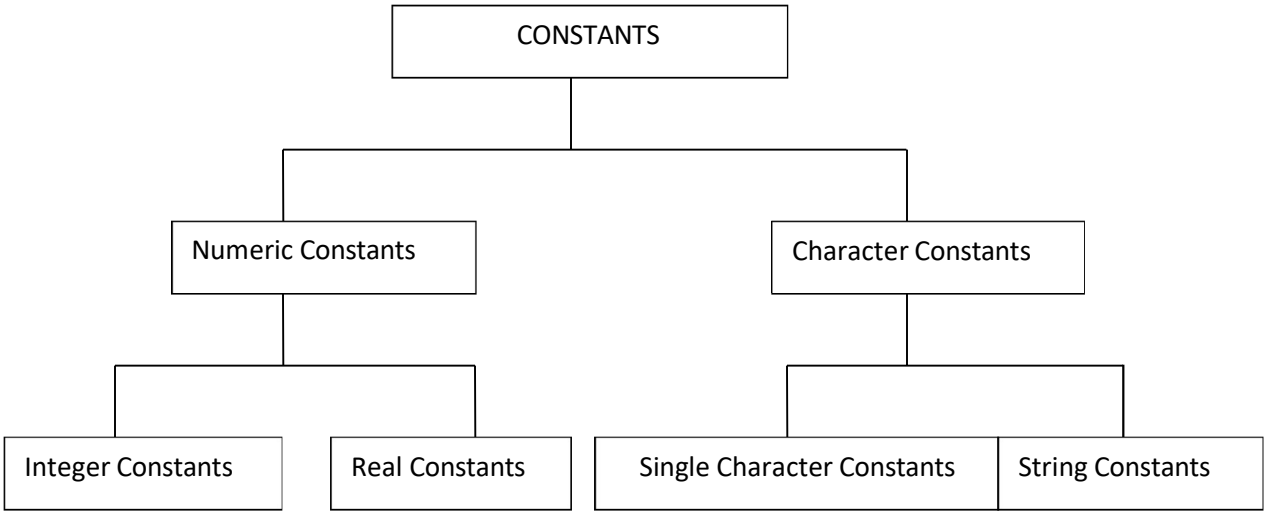
1. local variable
2. global variable
3. static variable

Constants

Constants **refer** to fixed values that do not change during the execution of a program.

Note: constants are also called literals.

C supports several kinds of constants.



TYPES OF C CONSTANT:

1. Integer constants
2. Real or Floating point constants
3. Character constants
4. String constants
5. Backslash character constants

Integer constants:

An integer constant is a numeric constant (associated with number) without any fractional or exponential part. There are three types of integer constants in C programming:

- decimal constant(base 10)
- octal constant(base 8)
- hexadecimal constant(base 16)

For example:

- Decimal constants: 0, -9, 22 etc
- Octal constants: 021, 077, 033 etc
- Hexadecimal constants: 0x7f, 0x2a, 0x521 etc

- In C programming, octal constant starts with a 0 and hexadecimal constant starts with a 0x.

1: Decimal Integer : the rules for represent decimal integer.

a) Decimal Integer value which consist of digits from 0-9.
b) Decimal Integer value with base 10.
c) Decimal Integer should not prefix with 0.
d) It allows only sign (+,-).
e) No special character allowed in this integer.

Ex :	**valid**	**invalid**
	7	$77
	77	077
	+77	7,777
	-77	

2 : Octal : An integer constants with base 8 is called octal. These rules are :

a) it consist of digits from 0 to 7.
b) It should prefix with 0.
c) It allows sign (+,-).
d) No special character is allowed.

EX :	**VALID**	**INVALID**
	0123	123 -> it because no prefix with 0
	+0123	0128 -> because digits from 0 to 7.
	-0123	

3 : Hexadecimal : An integer constant with base value 16 is called Hexadecimal.

a) It consist of digits from 0-9,a-f(capital letters & small leters.

Ex : 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

b) it should prefix with 0X or 0x.
c) it allows sign (+,-).
d) No special character is allowed.

EX : OX1a, ox2f

Floating point/Real constants:

A floating point constant is a numeric constant that has either a fractional form or an exponent form. For example:

-2.0

0.0000234

-0.22E-5

Note: E-5 = 10^{-5}

Real Constants : Real constant is base 10 number, which is represented in decimal 0r scientific/exponential notation.

Real Notation : The real notation is represented by an integer followed by a decimal point and the fractional(decimal) part. It is possible to omit digits before or after the decimal point.

Ex : 15.25
.75
30
-9.52
-92
+.94

***Scientific/Exponential Notation*:** The general form of Scientific/Exponential notation is

mantisha e exponent

The **mantisha** is either a real/floating point number expressed in decimal notation or an integer and the **exponent** is an integer number with an optional sign. The character **e** separating the mantisha and the exponent can be written in either lowercase or uppercase.

Ex : 1.5E-2
100e+3
-2.05e2

Character Constant:

Single Character Constant : A character constant is either a single alphabet, a single digit, a single special symbol enclosed within single inverted commas.

a) it is value represent in „ „ (single quote).
b) The maximam length of a character constant can be 1 character.

EX : VALID INVALID
„a“ “12”

„A“ „ab“

String constant : **A** string constant is a sequence of characters enclosed in double quote, the characters may be letters, numbers, special characters and blank space etc

EX : “rama” , “a” , “+123” , “1-/a”

"good" //string constant

"" //null string constant

" " //string constant of six white space

"x" //string constant having single character.

"Earth is round\n" //prints string with newline

Escape characters or backslash characters:

a) \n newline
b) \r carriage return
c) \t tab
d) \v vertical tab
e) \b backspace
f) \f form feed (page feed)
g) \a alert (beep)
h) \‟ single quote(„)
i) \” double quote(“)
j) \? Question mark (?)
k) \\ backslash (\)

Two ways to define constant in C

There are two ways to define constant in C programming.

1. const keyword
2. #define preprocessor
3.

1) C const keyword

The const keyword is used to define constant in C programming.

1. **const float** PI=3.14;

Now, the value of PI variable can't be changed.

```
1. #include <stdio.h>
2. #include <conio.h>
3. void main(){
4. const float PI=3.14;
5. clrscr();
6. printf("The value of PI is: %f",PI);
```

```
7.  getch();
8.  }
```
Output:
The value of PI is: 3.140000

2) C #define preprocessor

The #define preprocessor is also used to define constant.

C#define

The #define preprocessor directive is used to define constant or micro substitution. It can use any basic data type.

Syntax:

#define token value

Let's see an example of #define to define a constant.

```
#include <stdio.h>
1.  #define PI 3.14
2.  main() {
3.      printf("%f",PI);
4.  }
```
Output:
3.140000

Formatted and Unformatted Console I/O Functions.

Input / Output (I/O) Functions : In „C" language, two types of Input/Output functions are available, and all input and output operations are carried out through function calls. Several functions are available for input / output operations in „C". These functions are collectively known as the standard i/o library.

Input: In any programming language input means to feed some data into program. This can be given in the form of file or from command line.

Output: In any programming language output means to display some data on screen, printer or in any file.

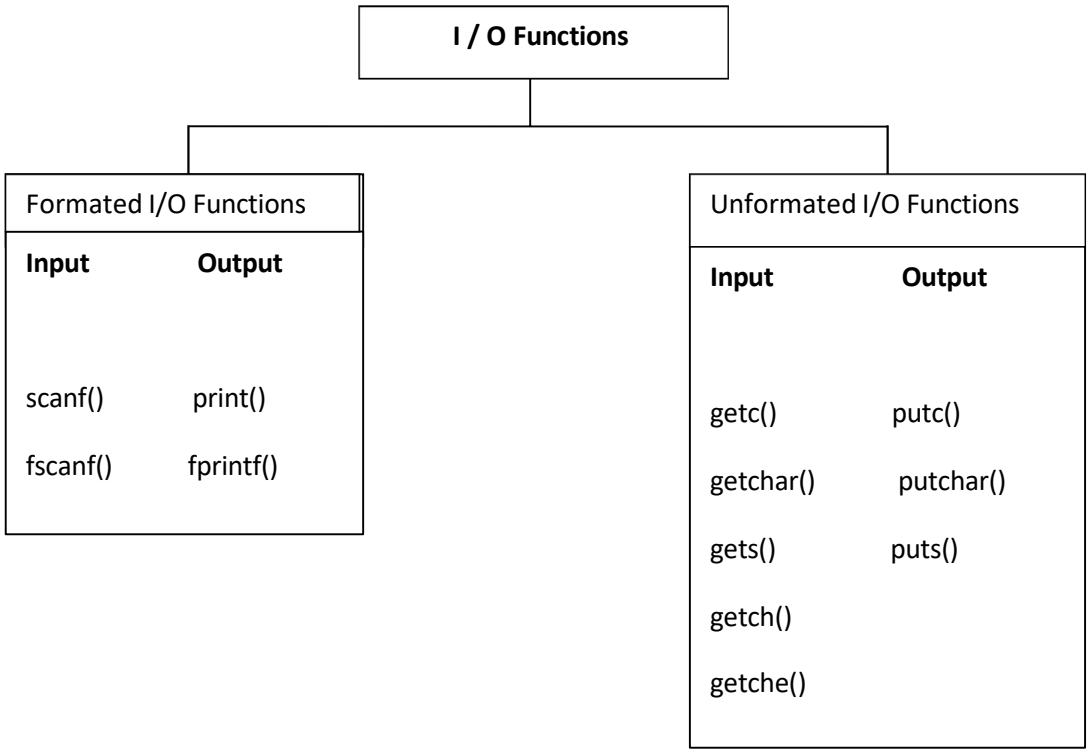
The Standard Files

C programming treats all the devices as files. So devices such as the display are addressed in the same way as files and the following three files are automatically opened when a program executes to provide access to the keyboard and screen.

Standard File	File Pointer	Device
Standard input	stdin	Keyboard

Standard output	stdout	Screen
Standard error	stderr	Your screen

Input / Output functions are classified into two types



. **Formated I/O Functions :** formatted I/O functions operates on various types of data.

1 : printf() : output data or result of an operation can be displayed from the computer to a standard output device using the library function printf(). This function is used to print any combination of data.

Syntax : printf("control string ", variable1, variable2, ----------, variablen);

Ex : printf("%d",3977); // **Output**: 3977

printf() statement another syntax :

Syntax : **printf(**"fomating string");

Formating string : it prints all the character given in doublequotes (" ") except formatting specifier.

Ex : printf(“ hello “);-> hello
printf(“a”); -> a
printf(“%d”, a); -> a value
printf(“%d”); -> no display

scanf() : input data can be entered into the computer using the standard input „C“ library function called scanf(). This function is used to enter any combination of input.

Syntax : scanf(“control string “,&var1, &var2, --- , &varn);

The scanf() function is used to read information from the standard input device (keyboard).

Ex : scanf(“ %d “,&a);-> hello

Each variable name (argument) must be preceeded by an ampersand (&). The (&) symbol gives the meaning “address of “ the variable.

Unformatted I/O functions:

a) **Character I/O**
b) **String I/O**

a) character I/O:

1. getchar(): Used to read a character from the standard input
2. putchar(): Used to display a character to standard output
3. getch() and getche(): these are used to take the any alpha numeric characters from the standard input
   getche() read and display the character
   getch() only read the single character but not display
4. putch(): Used to display any alpha numeric characters to standard output

a) String I/O:

1. gets(): Used for accepting any string from the standard input(stdin) eg:gets()

2. puts(): Used to display a string or character array Eg:puts()
3. Cgets():read a string from the console eg; cgets(char *st)
4. Cputs():display the string to the console eg; cputs(char *st)

OPERATORS AND EXPRESSIONS:

Operators : An operator is a Symbol that performs an operation. An operators acts some variables are called operands to get the desired result.

Ex : a+b;

Where a,b are operands and + is the operator.

Types of Operator :

1) Arithmetic Operators.
2) Relational Operators.
3) Logical Operators.
4) Assignment Operators.
5). Unary Operators.
6) Conditional Operators.
7) Special Operators.
8) Bitwise Operators.
9) Shift Operators.

Arithmetic Operators

An arithmetic operator performs mathematical operations such as addition, subtraction and multiplication on numerical values (constants and variables).

C Program to demonstrate the working of arithmetic operators

```
#include <stdio.h>
void  main()
{
   int a = 9,b = 4, c;

   c = a+b;
   printf("a+b = %d \n",c);

   c = a-b;
   printf("a-b = %d \n",c);

   c = a*b;
   printf("a*b = %d \n",c);

   c=a/b;
   printf("a/b = %d \n",c);

   c=a%b;
   printf("Remainder when a divided by b = %d \n",c);


}
```

Output
a+b = 13
a-b = 5
a*b = 36
a/b = 2
Remainder when a divided by b=1

Relational Operators. A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.
Operands may be variables, constants or expressions.

Relational operators are used in decision making and loops.

Operator	Meaning	Example	Return value
<	is less than	2<9	1
< =	is less than or equal to	2 < = 2	1
>	is greater than	2 > 9	0
> =	is greater than or equal to	3 > = 2	1
= =	is equal to	2 = = 3	0
!=	is not equal to	2!=2	0

```
// C Program to demonstrate the working of relational operators

#include <stdio.h>

int main()

{

    int a = 5, b = 5, c = 10;

    printf("%d == %d = %d \n", a, b, a == b); // true

    printf("%d == %d = %d \n", a, c, a == c); // false

    printf("%d > %d = %d \n", a, b, a > b); //false

    printf("%d > %d = %d \n", a, c, a > c); //false
```

```
    printf("%d < %d = %d \n", a, b, a < b); //false
    printf("%d < %d = %d \n", a, c, a < c); //true
    printf("%d != %d = %d \n", a, b, a != b); //false
    printf("%d != %d = %d \n", a, c, a != c); //true
    printf("%d >= %d = %d \n", a, b, a >= b); //true
    printf("%d >= %d = %d \n", a, c, a >= c); //false
    printf("%d <= %d = %d \n", a, b, a <= b); //true
    printf("%d <= %d = %d \n", a, c, a <= c); //true
    return 0;
}
```

Output

```
5 == 5 = 1
5 == 10 = 0
5 > 5 = 0
5 > 10 = 0
5 < 5 = 0
5 < 10 = 1
5 != 5 = 0
5 != 10 = 1
5 >= 5 = 1
5 >= 10 = 0
```

5 <= 5 = 1

5 <= 10 = 1

Logical Operators.

These operators are used to combine the results of two or more conditions. An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in decision making in C programming.

Operator	Meaning	Example	Return value
&&	Logical AND	(9>2)&&(17>2)	1
||	Logical OR	(9>2) || (17 = = 7)	1
!	Logical NOT	29!=29	0

Logical AND : If any one condition false the complete condition becomes false.

Truth Table

Op1	Op2	Op1 && Op2
true	true	true
true	false	false
false	true	false
false	false	false

Logical OR : If any one condition true the complete condition becomes true.

Truth Table

Op1	Op2	Op1 // Op2
true	true	true
true	false	true
false	true	true
false	false	false

Logical Not : This operator reverses the value of the expression it operates on i.e, it makes a true expression false and false expression true.

Op1	Op1 !
true	false
false	true

// C Program to demonstrate the working of logical operators

#include <stdio.h>

```
int main()
{
   int a = 5, b = 5, c = 10, result;
   result = (a = b) && (c > b);
   printf("(a = b) && (c > b) equals to %d \n", result);
   result = (a = b) && (c < b);
   printf("(a = b) && (c < b) equals to %d \n", result);
   result = (a = b) || (c < b);
   printf("(a = b) || (c < b) equals to %d \n", result);
   result = (a != b) || (c < b);
   printf("(a != b) || (c < b) equals to %d \n", result);
   result = !(a != b);
   printf("!(a == b) equals to %d \n", result);
   result = !(a == b);
   printf("!(a == b) equals to %d \n", result);
   return 0;
}
```

Output

```
(a = b) && (c > b) equals to 1
(a = b) && (c < b) equals to 0
(a = b) || (c < b) equals to 1
```

(a != b) || (c < b) equals to 0

!(a != b) equals to 1

!(a == b) equals to 0

Assignment Operators. Assignment operators are used to assign a value (or) an expression (or) a value of a variable to another variable.

Syntax : variable name=expression (or) value (or) variable

Ex : x=10;
y=a+b;
z=p;

Compound assignment operator:

„C" provides compound assignment operators to assign a value to variable in order to assign a new value to a variable after performing a specified operation.

Operator	Example	Meaning
+ =	x + = y	x=x+y
- =	x - = y	x=x-y
* =	x * = y	x=x*y
/ =	x / = y	x=x/y
% =	x % = y	X=x%y

// C Program to demonstrate the working of assignment operators

#include <stdio.h>

int main()

{

int a = 5, c;

c = a;

```
    printf("c = %d \n", c);
    c += a; // c = c+a
    printf("c = %d \n", c);
    c -= a; // c = c-a
    printf("c = %d \n", c);
    c *= a; // c = c*a
    printf("c = %d \n", c);
    c /= a; // c = c/a
    printf("c = %d \n", c);
    c %= a; // c = c%a
    printf("c = %d \n", c);
    return 0;
}
```

Output

```
c = 5
c = 10
c = 5
c = 25
c = 5
c = 0
```

Increment and Decrement Operators /Unary Operators:

Unary operators are having higher priority than the other operators. **Unary operators**, meaning **they only operate on a single operand.**

Increment Operator in C Programming

1. Increment operator is used to increment the current value of variable by adding integer 1.
2. Increment operator can be applied to only variables.
3. Increment operator is denoted by ++.

We have two types of increment operator i.e Pre-Increment and Post-Increment Operator**.**

Pre-Increment

Pre-increment operator is used to increment the value of variable before using in the expression. In the Pre-Increment value is first incremented and then used inside the expression.

b = ++y;

In this example suppose the value of variable „y" is 5 then value of variable „b" will be 6 because the value of „y" gets modified before using it in a expression**.**

Post-Increment

Post-increment operator is used to increment the value of variable as soon as after executing expression completely in which post increment is used. In the Post-Increment value is first used in a expression and then incremented.

b = x++;

In this example suppose the value of variable „x" is 5 then value of variable „b" will be 5 because old value of „x" is used.

Note :

We cannot use increment operator on the constant values because increment operator operates on only variables. It increments the value of the variable by 1 and stores the incremented value back to the variable

b = ++5;

or

b = 5++;

The **syntax** of the operators is given below.

++<variable name> --<variable name>
<variable name>++ <variable name>--

The operator ++ adds 1 to the operand and – subtracts 1 from the operand. These operators in two forms : prefix (++x) and postfix(x++).

Operator	Meaning
++x	Pre increment
- -x	Pre decrement
x++	Post increment
x--	Post decrement

Where
1 : ++x : Pre increment, first increment and then do the operation.
2 : - -x : Pre decrement, first decrements and then do the operation.
3 : x++ : Post increment, first do the operation and then increment.
4 : x- - : Post decrement, first do the operation and then decrement.

```
// C Program to demonstrate the working of increment and decrement operators
#include <stdio.h>
int main()
{
   int a = 10, b = 100;
   float c = 10.5, d = 100.5;
   printf("++a = %d \n", ++a);
   printf("--b = %d \n", --b);
   printf("++c = %f \n", ++c);
   printf("--d = %f \n", --d);
   return 0;
}
```

Output

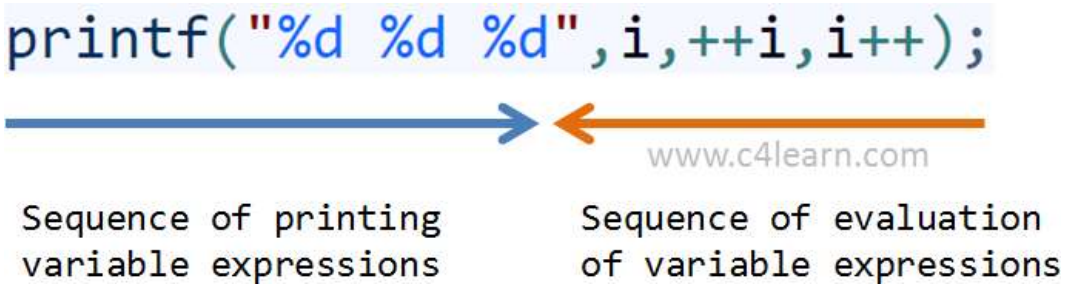
```
++a = 11
--b = 99
++c = 11.500000
++d = 99.500000
```

Multiple increment operators inside printf

```
#include<stdio.h>
void main() {
  int i = 1;
  printf("%d %d %d", i, ++i, i++);
}
```

Output : 3 3 1

Pictorial representation



Explanation of program

I am sure you will get confused after viewing the above image and output of program.

1. Whenever more than one format specifiers (i.e %d) are directly or indirectly related with same variable (i,i++,++i) then we need to evaluate each individual expression from right to left.
2. As shown in the above image evaluation sequence of expressions written inside printf will be – i++,++i,i
3. After execution we need to replace the output of expression at appropriate place

No	Step	Explanation
1	Evaluate i++	At the time of execution we will be using older value of i = 1
2	Evaluate ++i	At the time of execution we will be increment value already modified after step 1 i.e i = 3
2	Evaluate i	At the time of execution we will be using value of i modified in step 2

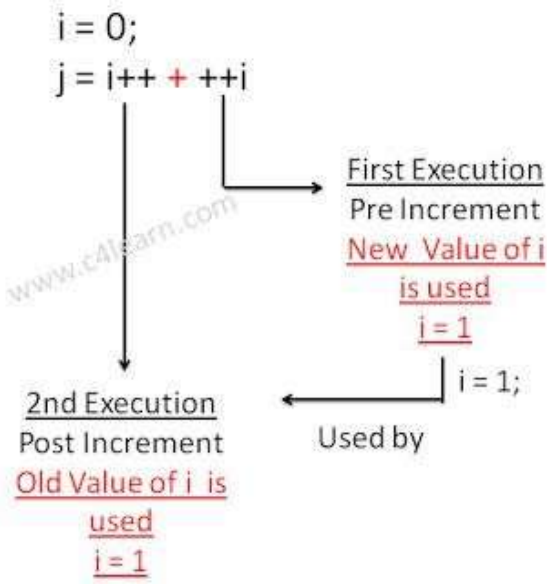
Postfix and Prefix Expression in Same Statement

```
#include<stdio.h>
#include<conio.h>
void main() {
   int i = 0, j = 0;
   j = i++ + ++i;
   printf("%d\n", i);
   printf("%d\n", j);
}
```

Output :

2
2

Explanation of Program



Conditional Operator/ Ternary operator:

conditional operator checks the condition and executes the statement depending of the condition.
A conditional operator is a ternary operator, that is, it works on 3 operands.
Conditional operator consist of two symbols.

1 : question mark (?).
2 : colon (:).

Syntax : condition ? exp1 : exp2;

It first evaluate the condition, if it is true (non-zero) then the "exp1" is evaluated, if the condition is false (zero) then the "exp2" is evaluated.

```
#include <stdio.h>
int main(){
  char February;
  int days;
  printf("If this year is leap year, enter 1. If not enter any integer: ");
  scanf("%c",&February);
  // If test condition (February == 'l') is true, days equal to 29.
  // If test condition (February =='l') is false, days equal to 28.
  days = (February == '1') ? 29 : 28;
  printf("Number of days in February = %d",days);
  return 0;
}
```

Output

If this year is leap year, enter 1. If not enter any integer: 1
Number of days in February = 29

Bitwise Operators:

Bitwise operators are used to manipulate the data at bit level. **It operates on integers only. It may not be applied to float.** In arithmetic-logic unit (which is within the CPU), mathematical operations like: addition, subtraction, multiplication and division are done in bit-level which makes processing faster and saves power. To perform bit-level operations in C programming, bitwise operators are used.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
<<	Shift left
>>	Shift right
~	One‟s complement.

Bitwise AND operator &

The output of bitwise AND is 1 if the corresponding bits of two operands is 1. If either bit of an operand is 0, the result of corresponding bit is evaluated to 0.

Let us suppose the bitwise AND operation of two integers 12 and 25.
12 = 00001100 (In Binary)
25 = 00011001 (In Binary)

Bit Operation of 12 and 25
 00001100
& 00011001

 ____
 00001000 = 8 (In decimal)

Example #1: Bitwise AND

```
#include <stdio.h>
int main()
{
   int a = 12, b = 25;
   printf("Output = %d", a&b);
   return 0;
}
```

Output

Output =8

Bitwise OR operator |

The output of bitwise OR is 1 if at least one corresponding bit of two operands is 1. In C Programming, bitwise OR operator is denoted by |.

12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

Bitwise OR Operation of 12 and 25

 00001100

| 00011001

 ____

 00011101 = 29 (In decimal)

Example #2: Bitwise OR

```
#include <stdio.h>

int main()

{

   int a = 12, b = 25;

   printf("Output = %d", a|b);

   return 0;
```

```
}
```

Output

Output =29

Bitwise XOR (exclusive OR) operator ^

The result of bitwise XOR operator is 1 if the corresponding bits of two operands are opposite. It is denoted by ^.

12 = 00001100 (In Binary)

25 = 00011001 (In Binary)

Bitwise XOR Operation of 12 and 25

```
  00001100
| 00011001
  ________
  00010101  = 21 (In decimal)
```

Example #3: Bitwise XOR

```
#include <stdio.h>

int main()

{

   int a = 12, b = 25;

   printf("Output = %d", a^b);

   return 0;

}
```

Output

Output = 21

Bitwise complement operator ~

Bitwise compliment operator is an unary operator (works on only one operand). It changes 1 to 0 and 0 to 1. It is denoted by ~.

35 = 00100011 (In Binary)

Bitwise complement Operation of 35

~ 00100011

_____

11011100 = 220 (In decimal)

Twist in bitwise complement operator in C Programming

The bitwise complement of 35 (~35) is -36 instead of 220, but why?

For any integer n, bitwise complement of n will be -(n+1). To understand this, you should have the knowledge of 2's complement.

2's Complement

Two's complement is an operation on binary numbers. The 2's complement of a number is equal to the complement of that number plus 1. For example:

Decimal	Binary	2's complement
0	**00000000**	**-(11111111+1) = -00000000 = -0(decimal)**
1	**00000001**	**-(11111110+1) = -11111111 = -256(decimal)**
12	**00001100**	**-(11110011+1) = -11110100 = -244(decimal)**
220	**11011100**	**-(00100011+1) = -00100100 = -36(decimal)**

Note: Overflow is ignored while computing 2's complement.

The bitwise complement of 35 is 220 (in decimal). The 2's complement of 220 is -36. Hence, the output is -36 instead of 220.

Bitwise complement of any number N is -(N+1). Here's how:

bitwise complement of N = ~N (represented in 2's complement form)

2'complement of ~N= -(~(~N)+1) = -(N+1)

Example #4: Bitwise complement

```
#include <stdio.h>
```

```
int main()

{

   printf("complement = %d\n",~35);

   printf("complement = %d\n",~-12);

   return 0;

}
```

Output

Complement = -36

Complement = 11

There are two Bitwise shift operators in C programming:

- Right shift operator
- Left shift operator.

Right Shift Operator

Right shift operator shifts all bits towards right by certain number of specified bits. It is denoted by >>.

Left Shift Operator

Left shift operator shifts all bits towards left by certain number of specified bits. It is denoted by <<.

Special Operators

1) Comma Operator :The comma operator is used to separate the statement elements such as variables, constants or expressions, and this operator is used to link the related expressions together, such expressions can be evaluated from left to right and the value of right most expressions is the value of combined expressions

Ex : val(a=3, b=9, c=77, a+c)

First signs the value 3 to a, then assigns 9 to b, then assigns 77 to c, and finaly80(3+77) to value.

2) Sizeof Operator : The sizeof() is a unary operator, that returns the length in bytes o the specified variable, and it is very useful to find the bytes occupied by the specified variable in the memory.

Syntax : **sizeof(variable-name);**

```
        int a;
Ex : sizeof(a);      //OUTPUT ---- 2bytes
```

Example #6: sizeof Operator

```
#include <stdio.h>
int main()
{
   int a, e[10];
   float b;
   double c;
   char d;
   printf("Size of int=%lu bytes\n",sizeof(a));
   printf("Size of float=%lu bytes\n",sizeof(b));
   printf("Size of double=%lu bytes\n",sizeof(c));
   printf("Size of char=%lu byte\n",sizeof(d));
   printf("Size of integer type array having 10 elements = %lu bytes\n", sizeof(e));
   return 0;
}
```

Output

```
Size of int = 4 bytes
Size of float = 4 bytes
Size of double = 8 bytes
Size of char = 1 byte
Size of integer type array having 10 elements = 40 bytes
```

Expressions

Expressions : An expression is a combination of operators and operands which reduces to a single value. An operator indicats an operation to be performed on data that yields a value. An operand is a data item on which an operation is performed.

A simple expression contains only one operator.

Ex : 3+5 is a simple expression which yields a value 8, -a is also a single expression.
A complex expression contain more than one operator.

Ex : complex expression is 6+8*7.

Ex ;	**Algeberic Expressions**	**C-expression**
	1 : ax2+bx+c	1: a*x*x+b*x+c
	2 : a+bx	2 : a+b*x.
	3 : 4ac/b	3 : 4*a*c/b.
	4 : x2/y2-1	4 : x*x/y*y-1

Operator Precedence : Arithmetic Operators are evaluvated left to right using the precedence of operator when the expression is written without the paranthesis.They are two levels of arithmetic operators in C.

1 : High Priority * / %
2 : Low Priority + -.

Arithmetic Expression evaluation is carried out using the two phases from left to right.

1 : First phase : The highest priority operator are evaluated in the 1st phase.
2 : Second Phase : The lowest priority operator are evaluated in the 2nd phase.

Ex : a=x-y/3+z*2+p/4.
x=7, y=9, z=11, p=8.
a= 7-9/3+11*2+8/4.

1st phase :
1 : a = 7-3+11*2+8/4
2 : a = 7-3+22+8/4
3 : a = 7-3+22+2
2nd phase :
1 : a = 4+22+2
2 : a = 26+2
3 : a = 28

The order of evaluation can be changed by putting paranthesis in an expression.
Ex : 9-12/(3+3)*(2-1)

Whenever parentheses are used, the expressions within parantheses highest priority. If two or more sets of paranthesis appear one after another. The expression contained in the left-most set is evaluated first and the right-most in the last.
1st phase :
1 : 9-12/6*(2-1)
2 : 9-12/6*1
2nd phase :
1 : 9-2*1
2 : 9-2.
3rd phase :

1 : 7.

Rules for Evaluation of Expression :

1 : Evaluate the sub-expression from left to right. If parenthesized.
2 : Evaluate the arithemetic Expression from left to right using the rules of precedence.
3 : The highest precedence is given to the expression with in paranthesis.
4 : When parantheses are used, the expressions within parantheses assume highest priority.
5 : Apply the associative rule, if more operators of the same precedence occurs.

Operator Precedence and Associativity :

Every operator has a precedence value. An expression containing more than one oerator is known as complex expression. Complex expressions are executed according to precedence of operators.

Associativity specifies the order in which the operators are evaluated with the same precedence in a complex expression. Associativity is of two ways, i.e left to ringht and right to left. Left to right associativity evaluates an expression starting from left and moving towards right. Right to left associativity proceds from right to left.

The precedence and associativity of various operators in C.

Operator	Description	Precedence	Associativity
( ) [ ]	Function call Square brackets.	1	L-R (left to right)
+ - ++ -- ! ~ * & sizeof	Unary plus Unary minus Increment Decrement Not operator Complement Pointer operator Address operator Sizeof operator	2	R-L (right to left)
* / %	Multiplication Division Modulo division	3	L-R (left to right)
+ -	Addition Subtraction	4	L-R (left to right)
<< >>	Left shift Right shift	5	L-R (left to right)

< <= > >=	Relational Operator	6	L-R (left to right)
= = !=	Equality Inequality	7	L-R (left to right)
&	Bitwise AND	8	L-R (left to right)
^	Bitwise XOR	9	L-R (left to right)
|	Bitwise OR	10	L-R (left to right)
&&	Logical AND	11	L-R (left to right)
||	Logical OR	12	L-R (left to right)
?:	Conditional	13	R-L (right to left)
= *= /= %= += -= &= ^= <<= >>=	Assignment operator	14	R-L (right to left)
,	Comma operator	15	L-R (left to right)

Type Conversion/Type casting:

Type conversion is used to convert variable from one data type to another data type, and after type casting complier treats the variable as of new data type.

For example, if you want to store a 'long' value into a simple integer then you can type cast 'long' to 'int'. You can convert the values from one type to another explicitly using the **cast operator.** Type conversions can be implicit which is performed by the compiler automatically, or it can be specified explicitly through the use of the cast operator.

Syntax:

(type_name) expression;

Without Type Casting:

```
1. int f= 9/4;
2. printf("f : %d\n", f );//Output: 2
```

With Type Casting:

```
1. float f=(float) 9/4;
2. printf("f : %f\n", f );//Output: 2.250000
```

Example:

```
#include <stdio.h>

int main()
```

```
{
   printf( "%c\n", (char)65 );
  getchar();
}
```

or

Type Casting - C Programming

Type casting refers to changing an variable of one data type into another. The compiler will automatically change one type of data into another if it makes sense. For instance, if you assign an integer value to a floating-point variable, the compiler will convert the int to a float. Casting allows you to make this type conversion explicit, or to force it when it wouldn‟t normally happen.

Type conversion in c can be classified into the following two types:

1. **Implicit Type Conversion**

When the type conversion is performed automatically by the compiler without programmers intervention, such type of conversion is known as implicit type conversion or type promotion.

```
int x;
for(x=97; x<=122; x++)
{
   printf("%c", x);   /*Implicit casting from int to char thanks to %c*/
}
```

2. **Explicit Type Conversion**

The type conversion performed by the programmer by posing the data type of the expression of specific type is known as explicit type conversion. The explicit type conversion is also known as type casting.

Type casting in c is done in the following form:

(data_type)expression;

where, data_type is any valid c data type, and expression may be constant, variable or expression.

For example,

```
int x;

for(x=97; x<=122; x++)

{

   printf("%c", (char)x);   /*Explicit casting from int to char*/

}
```

The following rules have to be followed while converting the expression from one type to another to avoid the loss of information:

All integer types to be converted to float.

All float types to be converted to double.

All character types to be converted to integer.

Example

Consider the following code:

```
int x=7, y=5 ;

float z;

z=x/y; /*Here the value of z is 1*/
```

If we want to get the exact value of 7/5 then we need explicit casting from int to float:

```
int x=7, y=5;

float z;

z = (float)x/(float)y;   /*Here the value of z is 1.4*/
```

Integer Promotion

Integer promotion is the process by which values of integer type "smaller" than int or unsigned int are converted either to int or unsigned int. Consider an example of adding a character with an integer −

```
#include <stdio.h>

main()

 {

   int  i = 17;

   char c = 'c'; /* ascii value is 99 */

   int sum;

   sum = i + c;

   printf("Value of sum : %d\n", sum );

}
```

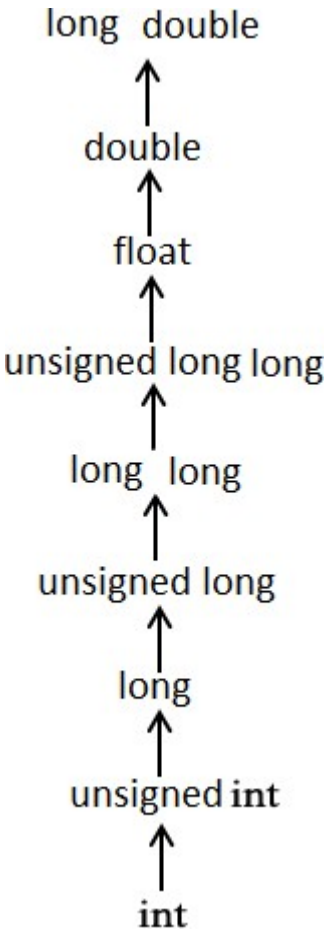
When the above code is compiled and executed, it produces the following result −

```
Value of sum : 116
```

Here, the value of sum is 116 because the compiler is doing integer promotion and converting the value of 'c' to ASCII before performing the actual addition operation.

Usual Arithmetic Conversion

The **usual arithmetic conversions** are implicitly performed to cast their values to a common type. The compiler first performs *integer promotion*; if the operands still have different types, then they are converted to the type that appears highest in the following hierarchy –



UNIT II

STATEMENTS

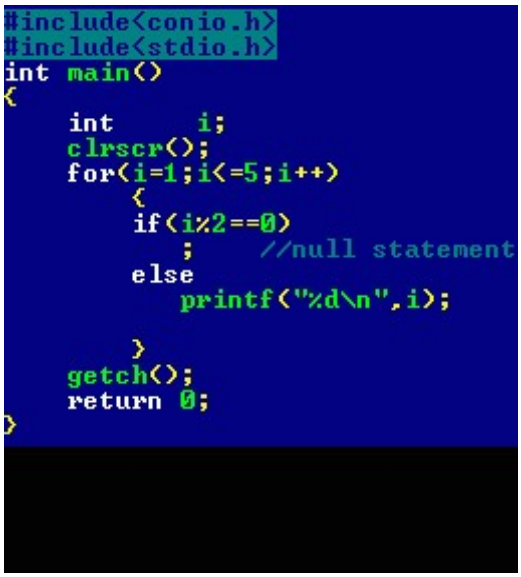
A statement causes the computer to carry out some definite action. There are three different classes of statements in C:

***Expression statements*, C*ompound statements*, and C*ontrol statements*.**

Null statement

A null statement consisting of only a semicolon and performs no operations. It can appear wherever a statement is expected. Nothing happens when a null statement is executed.

Syntax: - ;



```
#include<conio.h>
#include<stdio.h>
int main()
{
    int     i;
    clrscr();
    for(i=1;i<=5;i++)
        {
        if(i%2==0)
            ;    //null statement
        else
            printf("%d\n",i);

        }
    getch();
    return 0;
}
```

Statements such as **do**, **for**, **if**, and while require that an executable statement appear as the statement body. The null statement satisfies the syntax requirement in cases that do not need a substantive statement body.

The Null statement is nothing but, there is no body within loop or any other statements in C.

Example illustrates the null statement:

for (i = 0; i < 10; i++) ;

or

for (i=0;i<10;i++)

{

```
//empty body

}
```

Expression

Most of the statements in a C program are *expression statements*. An expression statement is simply an expression followed by a semicolon. The lines

```
i = 0;

i = i + 1;
```

and `printf("Hello, world!\n");`

are all expression statements. In C, however, the semicolon is a statement terminator. Expression statements do all of the real work in a C program. Whenever you need to compute new values for variables, you'll typically use expression statements (and they'll typically contain assignment operators). Whenever you want your program to do something visible, in the real world, you'll typically call a function (as part of an expression statement). We've already seen the most basic example: calling the function printf to print text to the screen.

Note -If no expression is present, the statement is often called the *null statement*.

Return

The return statement terminates execution of a function and returns control to the calling function, with or without a return value. A function may contain any number of return statements. The return statement has

syntax: return *expression*(opt);

If present, the expression is evaluated and its value is returned to the calling function. If necessary, its value is converted to the declared type of the containing function's return value.

A return statement with an expression cannot appear in a function whose return type is void . If there is no expression and the function is not defined as void , the return value is undefined. For example, **the following main function returns an unpredictable value to the operating system:**

```
main ( )

  {
```

return;

}

Compound statements

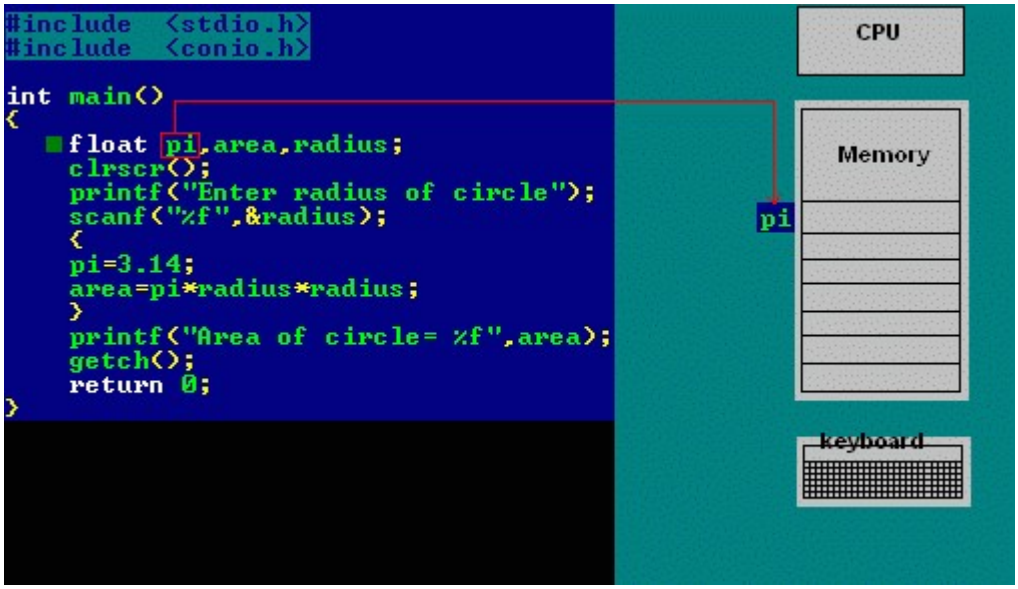
A compound statement (also called a "block") typically appears as the body of another statement, such as the if statement, for statement, while statement, etc

A Compound statement consists of several individual statements enclosed within a pair of braces { }. The individual statements may themselves be expression statements, compound statements or control statements. Unlike expression statements, a compound statement does not end with a semicolon. A typical Compound statement is given below.

```
{
pi=3.14;
area=pi*radius*radius;
}
```

The particular compound statement consists of two assignment-type expression statements.

Example:



Selection Statement/Conditional Statements/Decision Making Statements

A selection statement selects among a set of statements depending on the value of a controlling expression. Or

Moving execution control from one place/line to another line based on condition

Or

Conditional statements control the sequence of statement execution, depending on the value of a integer expression

C‟ language supports two conditional statements.

1: if

2: switch.

1: **if Statement:** The if Statement may be implemented in different forms.

1: simple if statement.

2: if –else statement

3: nested if-else statement.

4: else if ladder.

if statement.

The if statement controls conditional branching. The body of an if statement is executed if the value of the **expression is nonzero. Or if statement** is used to execute the code **if condition is true.** If the expression/condition is evaluated to false (0), statements inside the body of if is skipped from execution.

Syntax : if(condition/expression)

{

true statement;

}

statement-x;

If the condition/expression is true, then the true statement will be executed otherwise the true statement block will be skipped and the execution will jump to the statement-x. The „true statement" may be a single statement or group of statement.

If there is only one statement in the if block, then the braces are optional. But if there is more than one statement the braces are compulsory

Flowchart

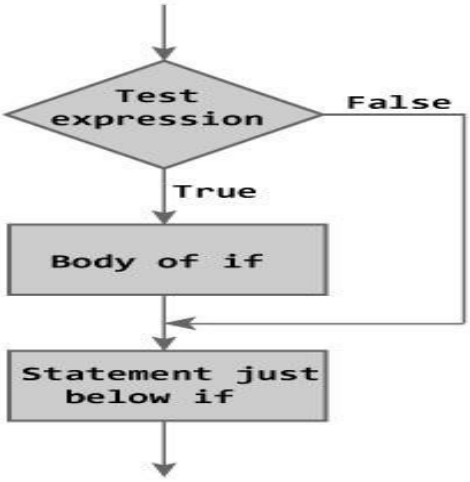


Figure: Flowchart of if Statement

Example:

```
#include<stdio.h>

main()

{

 int a=15,b=20;
```

```
 if(b>a)

{

printf("b is greater");

}

}
```

Output

```
b is greater
```

```
#include <stdio.h>
int main()
{
   int number;

   printf("Enter an integer: ");
   scanf("%d", &number);

   // Test expression is true if number is less than 0
   if (number < 0)
   {
      printf("You entered %d.\n", number);
   }

   printf("The if statement is easy.");

   return 0;
}
```

Output 1

```
Enter an integer: -2
You entered -2.
The if statement is easy.
```

Output 2

```
Enter an integer: 5
The if statement in C programming is easy.
```

If-else statement : The if-else statement is an extension of the simple if statement. The general form is. The if...else statement executes some code if the test expression is true (nonzero) and some other code if the test expression is false (0).

Syntax : if (condition)

```
{
    true statement;
}
else
{
    false statement;
}
statement-x;
```

If the condition is true , then the true statement and statement-x will be executed and if the condition is false, then the false statement and statement-x is executed.

Or

If test expression is true, codes inside the body of if statement is executed and, codes inside the body of else statement is skipped.

If test expression is false, codes inside the body of else statement is executed and, codes inside the body of if statement is skipped.

Flowchart

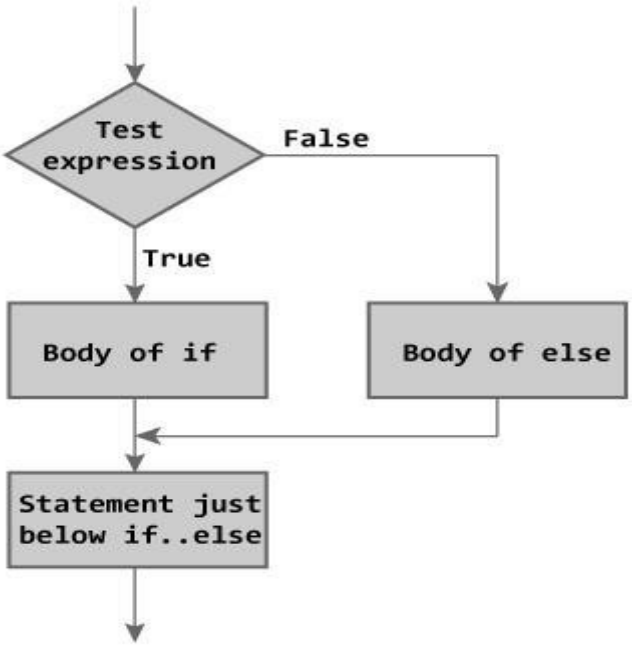


Figure: Flowchart of if...else Statement

Example:

// Program to check whether an integer entered by the user is odd or even

```
#include <stdio.h>
int main()
{
```

```
    int number;
    printf("Enter an integer: ");
    scanf("%d",&number);

    // True if remainder is 0
    if( number%2 == 0 )
        printf("%d is an even integer.",number);
    else
        printf("%d is an odd integer.",number);
    return 0;
}
```

Output

Enter an integer: 7
7 is an odd integer.

Nested if-else statement

When a series of decisions are involved, we may have to use more than on if-else statement in nested form. If –else statements can also be nested inside another if block or else block or both.

Syntax :

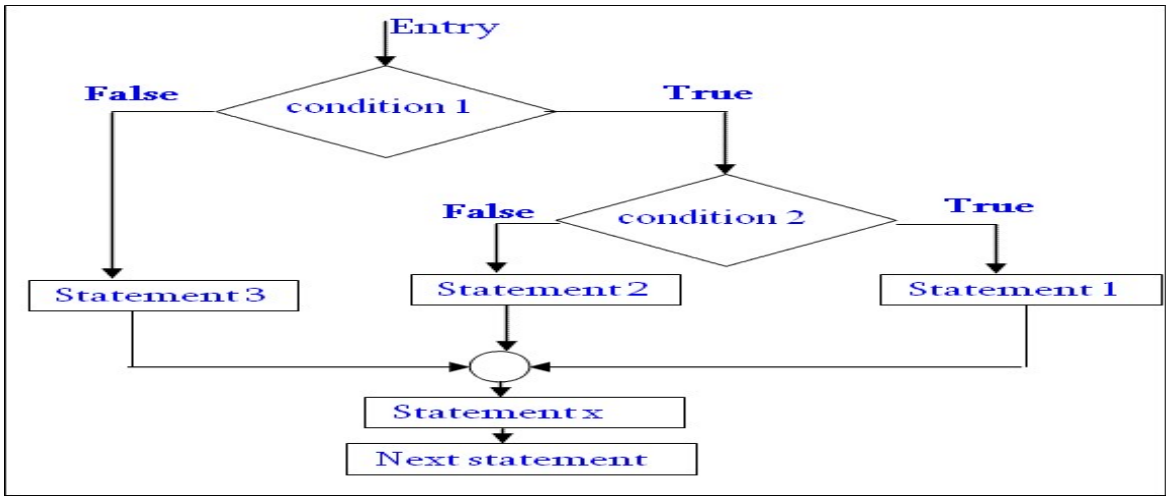
```
if(condition-1)
{                                          {
    if (condition-2)
    {
        statement-1;
    }
    else
    {
        statement-2;
    }
}
  else
  {
```

```
statement-3;
```

```
}
statement-x;
```

If the condition-1 is false, the statement-3 and statement-x will be executed. Otherwise it continues to perform the second test. If the condition-2 is true, the true statement-1 will be executed otherwise the statement-2 will be executed and then the control is transferred to the statement-x

Flowchart



Example

```
#include<stdio.h>
int var1, var2;
printf("Input the value of var1:");
scanf("%d", &var1);
printf("Input the value of var2:");
scanf("%d",&var2);
if (var1 !=var2)
{
    printf("var1 is not equal to var2");
    //Below – if-else is nested inside another if block
    if (var1 >var2)
    {
       printf("var1 is greater than var2");
    }
    else
    {
       printf("var2 is greater than var1");
    }
}
else
```

```
{
   printf("var1 is equal to var2");
}
…
```

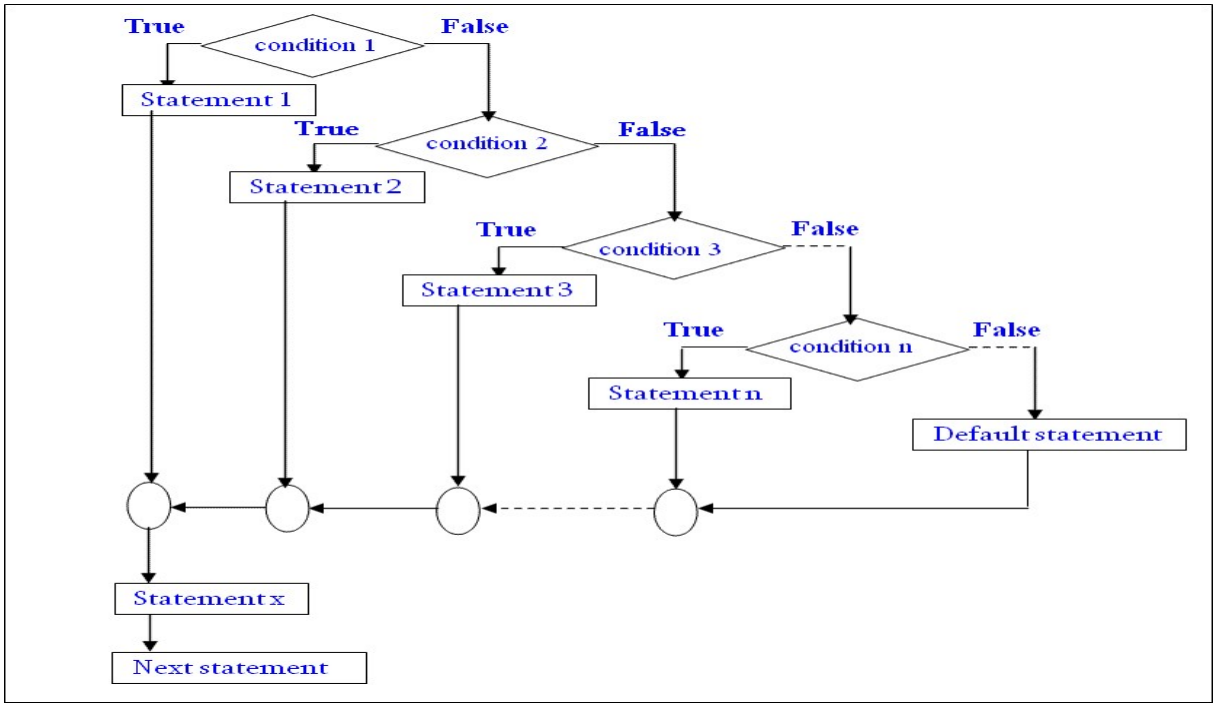
Else if ladder.

The if else-if statement is used to execute one code from multiple conditions**.**

Syntax :

```
if( condition-1)
    {
       statement-1;
    }
else if(condition-2)
   {
     statement-2;
    }
else if(condition-3)
   {
       statement-3;
    }
else if(condition-n)
  {
        statement-n;
    }
   else
     {
        default-statement;
      }
statement-x;
```

Flowchart



Example

```
#include<stdio.h>
 #include<conio.h>
 void main(){
 int number=0;
 clrscr();
  printf("enter a number:");
 scanf("%d",&number);
 if(number==10){
 printf("number is equals to 10");
 }
 else if(number==50){
 printf("number is equal to 50");
 }
 else if(number==100){
 printf("number is equal to 100");
 }
 else{
 printf("number is not equal to 10, 50 or 100");
 }
 getch();
 }
```

Points to Remember

1. In *if* statement, a single statement can be included without enclosing it into curly braces `{ }`

```
int a = 5;
if(a > 4)
  printf("success");
```

   No curly braces are required in the above case, but if we have more than one statement inside *if* condition, then we must enclose them inside curly braces.

5. `==` must be used for comparison in the expression of *if* condition, if you use `=` the expression will always return true, because it performs assignment not comparison.
6. Other than **0(zero)**, all other values are considered as true.

```
if(27)
  printf("hello");
```

   In above example, hello will be printed.

Switch statement : when there are several options and we have to choose only one option from the available ones, we can use switch statement. Depending on the selected option, a particular task can be performed. A task represents one or more statements.

Syntax:

```
switch(expression)
{
 case value-1:
        statement/block-1;
        break;
 case value-2:
        statement/block t-2;
        break;
 case value-3:
        statement/block -3;
        break;
 case value-4:
        statement/block -4;
        break;
 default:
        default- statement/block t;
        break;
```